

Les philosophes

Question 1 *Le dîner des philosophes*

Implémentez la solution du problème du dîner des philosophes discutée en cours. On utilisera les lock pour cette solution. Vérifiez qu'un interblocage est possible. Vérifiez que un philosophe pourrait ne jamais manger (d'où le terme *famine*)

Rappel utilisation de moniteurs

Chaque objet `ob` de Java a un *moniteur*. Pour prendre le moniteur il faut utiliser le mot clé `synchronized`.

```
synchronized (ob) {  
    ...code...  
}
```

Si un thread `t1` cherche à prendre un moniteur qui est déjà possédé par un autre thread, alors `t1` s'endort, et il sera réveillé quand le moniteur est disponible.

Tous les threads endormis sur un moniteur sont réveillés quand le moniteur devient disponible, mais un seul pourra le prendre, les autres s'endormiront à nouveau.

Si on veut utiliser le moniteur de `this`, et on veut le faire pour une méthode entière, alors on a une syntaxe particulière :

```
synchronized int f (int x){  
    ...code...  
}
```

Exercices

On utilisera maintenant les moniteurs à la place des locks.

Question 2 Écrire une application qui lance deux threads qui incrémentent chacun à son tour 10000 fois une variable partagée `x`, et qui ensuite affichent son contenu. En utilisant correctement les moniteurs, faites de façon que votre programme affiche toujours la valeur 20000. Une solution facile est d'exécuter un thread entièrement avant de lancer l'autre. Trouvez des solutions qui garantissent plus de concurrence.

Vous réaliserez cet exercice de deux façons :

- d'abord en utilisant le moniteur d'un objet de type `Object`
- ensuite en écrivant une méthode qui utilise le moniteur de `this`

Question 3 Faire la même chose que l'exercice précédent, mais avec 50 threads

Question 4 *L'aéroport*

Des avions souhaitent décoller et atterrir et demandent l'accès à l'unique piste. Une tour de contrôle doit gérer cette piste. Quand un avion souhaite décoller ou atterrir, il attend l'autorisation de la tour de contrôle. Il entre sur la piste. Si jamais deux avions sont sur la piste en même temps un *crash se produit*.

Les avions sont des threads java.

Ecrivez les classes **Airplane**, **Airport**, **ControlTower**.

Dans un premier temps on fera une solution sans synchronisation, et on observera les crashes. Comment peut-on savoir s'il y a plusieurs avions ?

Par la suite vous vous synchroniserez sur la tour de contrôle pour éviter les crashes.

(En passant, pour garantir la synchronisation, on pourrait écrire le code de façon qu'une seule tour de contrôle puisse être créée. Cela s'appelle le pattern "Singleton" : si cela vous intéresse, cherchez comment il s'implémente en Java, et faites-le)